CPU Simulator
Implementation Report

Computer Architecture · Project #1

단국대학교 모바일시스템공학과

학번: 32240410

이름: 김나경

Left freedays: 5days

Dankook University · Dept. of Mobile Systems Engineering
Instructor: Prof. Seehwan Yoo

Language: Python 3.10 · Platform: macOS · No external libraries

1. Project Overview

본 프로젝트는 간단한 ISA(Instruction Set Architecture)를 직접 설계하고, Von Neumann 실행 모델에 기반한 CPU 시뮬레이터를 Python 으로 구현하는 과제이다. 명령어 해석, 레지스터 및 메모리 상태 변화, 프로그램 흐름 제어를 직접 구현함으로써 컴퓨터 구조의 핵심 원리를 실습 수준에서 체득하는 것을 목표로 한다.

File Structure

```
HW1
├── assembler/
│   ├── assembler.py      ← Assembly 전체를 Instruction 리스트로 변환
│   └── parser.py         ← 명령어 한 줄 파싱 및 operand 타입 판별
├── devices/
│   ├── memory_unit/
│   │   └── memory.py     ← Instruction 저장용 메모리 모델
│   ├── processing_unit/
│   │   └── flags.py      ← Zero / Negative 상태 관리
│   └── register_unit/
│       ├── ir.py         ← 현재 실행 Instruction 저장 (IR)
│       ├── pc.py         ← 다음 실행 주소 관리 (PC)
│       └── register_file.py ← R0~R9 범용 레지스터
├── execution/
│   ├── cpu.py            ← Fetch–Decode–Execute 실행 루프
│   └── loader.py         ← Instruction 을 Memory 에 적재
├── isa/
│   └── instruction.py    ← Instruction 객체 및 ISA 정의
├── tests/                ← 13 개 테스트 케이스 (.txt)
└── main.py               ← 테스트 프로그램 실행 관리
```

File Description

File	Description
assembler.py	Assembly 소스 전체를 파싱하여 Instruction 객체 리스트로 변환. Label 테이블을 2-pass 로 생성하고 분기 주소를 사전 해석.
parser.py	명령어 한 줄을 Opcode / Operand 로 분리. 0x 접두사(hex), R 접두사(레지스터), 정수(immediate)를 자동 판별.
memory.py	주소 기반 Instruction 저장 공간. read(addr) / write(addr, val) 인터페이스 제공.
flags.py	Zero Flag / Negative Flag 상태 관리. 산술 연산 결과에 따라 update_from_result()로 갱신.

ir.py	Fetch 단계에서 읽어온 Instruction 을 보관하는 Instruction Register.
pc.py	get() / set(addr) / increment() 인터페이스를 제공하는 Program Counter.
register_file.py	R0~R9 범용 레지스터. read/write 시 이름 및 타입 유효성 검사 포함.
cpu.py	Fetch → Decode → Execute 사이클을 HALT 까지 반복하는 메인 실행 루프.
loader.py	Instruction 리스트를 Memory 주소 0 부터 순서대로 적재.
instruction.py	Instruction dataclass 및 Opcode 상수 정의. ISA 전체 명령어 집합 관리.
main.py	tests/ 디렉토리의 모든 테스트 파일을 순차 실행하고 결과 출력.

2. Introduction

본 과제의 목표는 간단한 Instruction Set Architecture 를 기반으로 CPU 실행 과정을 소프트웨어적으로 구현하는 것이다. 명령어 해석, 프로그램 흐름 제어, 레지스터 및 메모리 상태 변화 과정을 직접 구현함으로써 컴퓨터 구조의 핵심 원리를 이해하는 것이 목적이다.

특히 Program Counter 기반의 순차 실행 구조를 구현함으로써 Stored Program Concept 과 Von Neumann Execution Model 을 실제 동작 수준에서 확인하는 데 중점을 두었다. 각 구성 요소를 독립적인 모듈로 분리하여 Separation of Concerns 원칙을 따르는 설계를 지향하였다.

3. Work Description

본 프로젝트에서는 다음 기능을 구현하였다.

- Assembly Parser 및 2-pass Label Resolution
- Instruction Memory 구성 및 Loader
- Program Counter 기반 Sequential Execution
- Arithmetic Instructions 실행: ADD, SUB, MUL, DIV
- Comparison 및 Flag 기반 Branch 처리: CMP, JZ, JN
- 레지스터 기반 Conditional Branch: C, B
- Unconditional Jump: J

- Division by Zero Runtime Error 처리
- Instruction Register 도입 — Fetch / Execute 단계 구조적 분리
- GCD 반복 알고리즘 실행 및 전체 테스트 케이스 검증

CPU 실행은 다음 6 단계로 구성된다.

1. Program Counter 가 가리키는 주소에서 Instruction Fetch
2. Instruction Register 에 로드
3. Opcode Decode
4. Operand Resolve (레지스터 참조 또는 즉시값 해석)
5. Register / Flag 상태 변경 (Execute)
6. Program Counter 갱신 (순차 증가 또는 분기 주소 설정)

4. ISA Design

본 프로젝트에서 사용한 ISA 는 아래 표와 같다. 명령어 형식은 'Opcode Operand1, Operand2' 이며, operand 는 레지스터(R0~R9), 10 진 정수, 0x 접두 16 진수, 또는 Label 중 하나이다.

Category	Instruction	동작 설명
Data	MOV Rd, Src	Src(레지스터 or 즉시값)를 Rd 에 저장
Arithmetic	ADD Rd, Rs	Rd + Rs → R0
	SUB Rd, Rs	Rd - Rs → R0
	MUL Rd, Rs	Rd × Rs → R0
	DIV Rd, Rs	Rd ÷ Rs → R0 (정수 나눗셈, 0 으로 나누면 RuntimeError)
Comparison	CMP Rd, Rs	Rd - Rs 결과로 Zero / Negative Flag 갱신
	C Rd, Rs	Rd < Rs 이면 R0 ← 1, 아니면 R0 ← 0
Control	J target	target 주소로 무조건 점프
	JZ target	Zero Flag == 1 이면 target 으로 점프
	JN target	Negative Flag == 1 이면 target 으로 점프
	B target	R0 == 1 이면 target 으로 조건 분기
Termination	HALT	실행 종료, 레지스터 상태 출력

특히 C 와 B 명령어는 슬라이드의 의미에 맞게 구현하였다.

- C: operand1 < operand2 이면 R0 ← 1, 아니면 R0 ← 0
- B: R0 == 1 일 때만 target 주소로 분기 (R0 가 0 이면 다음 명령어로 진행)

이는 비교 결과를 레지스터 기반 조건으로 처리하는 Control 모델이다. CMP 는 Flags 를 갱신하여 JZ / JN 에서, C 는 R0 를 갱신하여 B 에서 활용한다.

instruction.py — ISA 정의

Instruction 은 Python dataclass 로 정의하여 opcode 와 operands 를 타입 안전하게 관리하였고, Opcode 클래스에 전체 명령어 집합(ALL)을 정의하여 파서에서 유효성 검사에 활용하였다.

```
# instruction.py
@dataclass
class Instruction:
    opcode: str
    operands: List[Operand] = field(default_factory=list)

    def __post_init__(self):
        self.opcode = self.opcode.upper()

class Opcode:
    MOV = 'MOV';  ADD = 'ADD';  SUB = 'SUB'
    MUL = 'MUL';  DIV = 'DIV';  CMP = 'CMP'
    C   = 'C';    J   = 'J';    JZ  = 'JZ'
    JN  = 'JN';   B   = 'B';    HALT = 'HALT'
    ALL = { MOV, ADD, SUB, MUL, DIV, CMP, C, J, JZ, JN, B, HALT }
```

5. Microarchitecture

CPU 내부 구성 요소는 다음과 같이 각자 독립된 모듈로 분리하여 구현하였다.

- **Memory**: 주소 기반 Instruction 저장 공간. Loader 가 프로그램을 순서대로 적재한다.
- **Program Counter (pc.py)**: get() / set(addr) / increment() 인터페이스로 실행 주소를 관리.
- **Instruction Register (ir.py)**: Fetch 한 Instruction 을 보관. load() / read() 로 Fetch 와 Execute 를 구조적으로 분리.
- **Register File (register_file.py)**: R0~R9 10 개 범용 레지스터. 쓰기 시 타입 및 이름 유효성 검사 포함.

- **Flags (flags.py)**: Zero / Negative 두 Flag 를 관리. update_from_result(result)로 CMP 결과를 반영.
- **Execution Loop (cpu.py)**: HALT 또는 예외 발생 시까지 Fetch→Decode→Execute 사이클을 반복.

assembler.py — 2-pass Label Resolution

Label 을 지원하기 위해 assembler.py 에서 2-pass 방식을 구현하였다. 1st pass 에서 Label 테이블을 생성하고, 2nd pass 에서 분기 명령어의 operand 를 실제 주소로 치환한다.

```
# assembler.py
def assemble(program_text: str):
    lines = program_text.split('\n')

    # 1st pass: Label 테이블 생성
    labels = {}
    instruction_index = 0
    for line in lines:
        stripped = line.strip()
        if not stripped: continue
        if stripped.endswith(':'):
            labels[stripped[:-1]] = instruction_index
            continue
        instruction_index += 1

    # 2nd pass: 분기 operand 를 주소로 치환
    instructions = []
    for line in lines:
        stripped = line.strip()
        if not stripped or stripped.endswith(':'): continue
        inst = parse_line(stripped)
        if inst.opcode in {'J', 'JZ', 'JN', 'B'}:
            target = inst.operands[0]
            if isinstance(target, str) and target in labels:
                inst.operands[0] = labels[target]
        instructions.append(inst)
    return instructions
```

GCD.txt 의 경우 LOOP, LESS, END 등의 Label 이 사용되며, assembler 는 이를 각 주소(2, 8, 11)로 자동 변환한다.

cpu.py — Fetch–Decode–Execute 루프

cpu.py 의 run() 메서드는 전체 실행 사이클을 담당한다. 분기 명령어(J / JZ / JN / B)는 PC 를 직접 갱신한 뒤 continue 로 increment()를 건너뛰고, 그 외 명령어는 루프 말미의 pc.increment()로 순차 진행한다.

```
# cpu.py
def run(self):
    while True:
        address = self.pc.get()
        fetched = self.memory.read(address)   # Fetch
        self.ir.load(fetched)                 # IR 에 로드
        instr = self.ir.read()
        print(f'[PC={address}] Executing: {instr}')

        opcode = instr.opcode
        operands = instr.operands

        if opcode == Opcode.ADD:
            v1 = self.resolve_operand(operands[0])
            v2 = self.resolve_operand(operands[1])
            self.registers.write('R0', v1 + v2)

        elif opcode == Opcode.JZ:
            target = self.resolve_operand(operands[0])
            if self.flags.zero:
                self.pc.set(target)
                continue          # ← increment() 건너뜀

        elif opcode == Opcode.HALT:
            print(self.registers.dump())
            break

        self.pc.increment()       # 순차 실행

    def resolve_operand(self, operand):
        if isinstance(operand, int): return operand
        elif isinstance(operand, str): return self.registers.read(operand)
        else: raise ValueError(f'Invalid operand: {operand}')
```

6. Execution Example

ADD Program

가장 기본적인 산술 연산 테스트이다. R1 = 3, R2 = 7 을 설정하고 ADD 연산으로 두 값의 합을 R0 에 저장한다.

입력 (ADD.txt):

```
MOV R1, 3
MOV R2, 7
ADD R1, R2
HALT
```

실행 결과:

```
---------------
Running ADD.txt
---------------
[PC=0] Executing: MOV R1, 3
[PC=1] Executing: MOV R2, 7
[PC=2] Executing: ADD R1, R2
[PC=3] Executing: HALT
{'R0': 10, 'R1': 3, 'R2': 7, 'R3': 0, 'R4': 0, 'R5': 0, 'R6': 0, 'R7': 0, 'R8': 0, 'R9': 0}
```

PC 가 0→1→2→3 으로 순차 증가하며 Von Neumann 순차 실행이 정상 동작함을 보여준다. ADD 연산 결과(3+7=10)는 R0 에 저장된다.

Branch (C / B) 테스트

C 명령어로 두 레지스터를 비교하고, B 명령어로 조건 분기를 수행한다. 두 케이스로 분기가 taken / not-taken 모두 올바른지 검증하였다.

B.txt — 분기 발생 (R1=1 < R2=3 이므로 R0=1 → 분기):

```
---------------
Running B.txt
---------------
[PC=0] Executing: MOV R1, 1
[PC=1] Executing: MOV R2, 3
[PC=2] Executing: C R1, R2
[PC=3] Executing: B 6
[PC=6] Executing: MOV R3, 1
[PC=7] Executing: HALT
{'R0': 1, 'R1': 1, 'R2': 3, 'R3': 1, 'R4': 0, 'R5': 0, 'R6': 0, 'R7': 0, 'R8': 0, 'R9': 0}
```

B_not_taken.txt — 분기 미발생 (R1=7 >= R2=3 이므로 R0=0 → 분기 안 함):

```
---------------
Running B_not_taken.txt
---------------
[PC=0] Executing: MOV R1, 7
[PC=1] Executing: MOV R2, 3
[PC=2] Executing: C R1, R2
[PC=3] Executing: B 6
[PC=4] Executing: MOV R3, 9
[PC=5] Executing: J 7
[PC=7] Executing: HALT
{'R0': 0, 'R1': 7, 'R2': 3, 'R3': 9, 'R4': 0, 'R5': 0, 'R6': 0, 'R7': 0, 'R8': 0, 'R9': 0}
```

Division by Zero 예외 처리

DIV 명령어 실행 시 제수가 0 이면 ZeroDivisionError 를 발생시키고, main.py 에서 이를 catch 하여 사용자에게 명확한 오류 메시지를 출력한다.

입력 (DIV_zero.txt) / 실행 결과:

```
---------------
Running DIV_zero.txt
---------------
[PC=0] Executing: MOV R1, 10
[PC=1] Executing: MOV R2, 0
[PC=2] Executing: DIV R1, R2
Runtime Error: Division by zero
```

cpu.py DIV 구현 발췌:

```
elif opcode == Opcode.DIV:
    v1 = self.resolve_operand(operands[0])
    v2 = self.resolve_operand(operands[1])
    if v2 == 0:
        raise ZeroDivisionError('Division by zero')
    self.registers.write('R0', v1 // v2)
```

GCD Program

GCD 알고리즘은 본 프로젝트의 핵심 검증 케이스이다. CMP 로 두 레지스터를 비교하고, JZ / JN 으로 분기하며, SUB 로 큰 수에서 작은 수를 반복해서 빼는 방식으로 GCD 를 계산한다.

입력 (GCD.txt) — GCD(12, 8) = 4:

```
MOV R1, 12
MOV R2, 8
LOOP:
  CMP R1, R2
  JZ  END        ; R1 == R2 이면 종료
  JN  LESS       ; R1 < R2 이면 LESS 로 분기
  SUB R1, R2     ; R1 > R2: R1 = R1 - R2
  MOV R1, R0
  J   LOOP
LESS:
  SUB R2, R1     ; R1 < R2: R2 = R2 - R1
  MOV R2, R0
  J   LOOP
END:
  HALT
```

실행 결과:

```
---------------
Running GCD.txt
---------------
[PC=0] Executing: MOV R1, 12
[PC=1] Executing: MOV R2, 8
[PC=2] Executing: CMP R1, R2
[PC=3] Executing: JZ 11
[PC=4] Executing: JN 8
[PC=5] Executing: SUB R1, R2
[PC=6] Executing: MOV R1, R0
[PC=7] Executing: J 2
[PC=2] Executing: CMP R1, R2
[PC=3] Executing: JZ 11
[PC=4] Executing: JN 8
[PC=8] Executing: SUB R2, R1
[PC=9] Executing: MOV R2, R0
[PC=10] Executing: J 2
[PC=2] Executing: CMP R1, R2
[PC=3] Executing: JZ 11
[PC=11] Executing: HALT
{'R0': 4, 'R1': 4, 'R2': 4, 'R3': 0, 'R4': 0, 'R5': 0, 'R6': 0, 'R7': 0, 'R8': 0, 'R9': 0}
```

3 회 반복으로 GCD(12, 8) = 4 가 정확히 계산되었다. Branch 와 Jump 흐름이 올바르게 동작함을 확인하였다.

전체 테스트 결과 요약

Test File	검증 내용	결과
ADD.txt	R1=3, R2=7 → R0=10	PASS ✓
MUL.txt	R1=6, R2=7 → R0=42	PASS ✓
DIV.txt	R1=20, R2=4 → R0=5	PASS ✓
DIV_zero.txt	0 으로 나누기 → RuntimeError	PASS ✓
MOV.txt	레지스터 간 MOV	PASS ✓
MUL_DIV_mix.txt	8×3=24, 24÷2=12 복합 연산	PASS ✓
C_true.txt	R1=2 < R2=5 → R0=1	PASS ✓
C_false.txt	R1=5 >= R2=2 → R0=0	PASS ✓
C_equal.txt	R1=5 == R2=5 → R0=0	PASS ✓
B.txt	R0=1 → 분기 발생	PASS ✓
B_not_taken.txt	R0=0 → 분기 미발생	PASS ✓
JZ.txt	Zero Flag → JZ 분기	PASS ✓
JN.txt	Negative Flag → JN 분기	PASS ✓
GCD.txt	GCD(12, 8) = 4	PASS ✓

7. Reflection

최근 데이터 파이프라인을 설계하는 작업을 하면서 일정한 조건과 실행 흐름만 명확히 정의해 두면 이후 시스템은 사람의 개입 없이 정해진 규칙에 따라 자동으로 동작한다는 경험을 한 적이 있다. 이번 CPU 시뮬레이터 구현 과제를 진행하면서 프로그램 실행 역시 매우 유사한 원리 위에 구성되어 있다는 느낌을 받았다. 시스템은 스스로 판단하지 않는다. 사람이 먼저 구조와 규칙을 설계하면 이후의 동작은 상태 변화의 반복으로 이어진다. 이러한 관점에서 프로그램 실행이라는 개념이 생각보다 단순한 구조적 원리 위에 구축되어 있다는 점이 인상적으로 다가왔다.

이번 과제를 통해 가장 분명하게 이해하게 된 부분은 컴퓨터 실행 모델의 핵심이 Von Neumann 구조에 있다는 사실이었다. Program Counter 가 다음 실행할 명령어를 결정하고 CPU 는 해당 위치의 Instruction 을 순차적으로 읽고 실행한다. 실행 흐름은 기본적으로 선형적이며 분기 명령어가 등장할 때만 경로가 변경된다. 이러한 단순한 실행 원리가 오늘날 대부분의 범용 컴퓨터 시스템의 기반이라는 점을 실제 구현을 통해 체감할 수 있었다.

또한 Von Neumann 모델과 Dataflow 모델을 비교하면서 병렬성에 대한 고민도 자연스럽게 이어졌다. 순차 실행 구조는 제어 흐름이 명확하여 이해와 구현이 수월하지만 실행 단위가 Program Counter 중심으로 강하게 제한되기 때문에 자연스러운 병렬 실행을 끌어내기 어렵다. 반면 Dataflow 모델은 데이터 준비 여부에 따라 연산이 수행될 수 있어 높은 병렬성을 기대할 수 있지만 제어 구조가 복잡해지고 시스템 설계 난이도 또한 증가한다. 이러한 차이를 통해 컴퓨터 구조 설계는 실행의 단순성, 프로그래밍의 용이성, 성능 향상 사이에서 균형점을 찾는 과정이라는 생각을 하게 되었다.

Instruction Set Architecture 와 Microarchitecture 의 관계 역시 구현 과정에서 보다 현실적으로 체감할 수 있었다. 이번 과제에서는 사용할 ISA 가 일정 부분 주어져 있었지만 이를 실제 실행 구조로 구현하는 방식은 각자의 선택에 맡겨져 있었다. 같은 명령어 의미를 기반으로 하더라도 내부 상태 관리 방식이나 실행 흐름 구성에 따라 서로 다른 구조가 만들어질 수 있다는 점이 흥미롭게 느껴졌다. 어쩌면 교수님께서 구현 과제를 제시한 것 자체가 이러한 설계 다양성과 구조적 사고를 경험하게 하기 위한 의도가 아니었을까 하는 생각도 들었다. 같은 기능을 수행하는 시스템이라도 내부 구현 방식은 전혀 다르게 설계될 수 있으며, 바로 그 지점에서 Microarchitecture 설계의 의미가 드러난다는 점을 직접 체감할 수 있었다.

이번 과제를 통해 컴퓨터 구조는 단순히 하드웨어 내부 동작을 배우는 분야가 아니라 프로그램 실행이라는 현상을 구조적으로 이해하는 학문이라는 인식을 갖게 되었다. 사람이 사고를 통해 실행 규칙과 구조를 설계하고 시스템은 그 위에서 상태 변화를 반복한다는 관점은 프로그램 실행의 구조를 직접 구성해본 경험은 이후 어떤 시스템을

바라볼 때에도 그 내부에서 어떤 상태 변화가 반복되고 있는지를 자연스럽게 상상하게 만드는 계기가 될 것이라고 생각한다.

8. Build Environment

항목	내용
Language	Python 3.10
OS	macOS
외부 라이브러리	없음 (표준 라이브러리만 사용)
실행 방법	python main.py
테스트 실행	tests/ 내 모든 .txt 파일 자동 순회
테스트 케이스	14 개 (ADD, SUB, MUL, DIV, DIV_zero, MOV, MUL_DIV_mix, C_true, C_false, C_equal, B, B_not_taken, JZ, JN, GCD)

9. Conclusion

본 프로젝트에서는 간단한 Instruction Set Architecture 를 기반으로 CPU 실행 모델을 직접 구현하고, 프로그램이 메모리에서 읽혀 실행되는 과정을 구조적으로 확인하였다. Program Counter, Instruction Register, Register File, Flags, Memory 와 같은 구성 요소를 설계하고 Fetch–Decode–Execute 흐름을 구현함으로써 명령어 실행의 기본 원리를 체계적으로 이해할 수 있었다.

특히 2-pass Assembler 구현을 통해 Label 기반의 분기 주소를 자동 해석하는 과정에서, 실제 어셈블러가 소스 코드를 기계어로 변환하는 원리를 직접 체험할 수 있었다. GCD 알고리즘의 정확한 실행과 전체 14 개 테스트 케이스의 통과를 통해 시뮬레이터의 정확성을 검증하였다.

또한 동일한 ISA 를 기반으로 하더라도 Microarchitecture 설계 방식에 따라 시스템의 동작 구조가 달라질 수 있다는 점을 체감할 수 있었으며, 이러한 설계 선택이 성능, 복잡도, 확장 가능성과 밀접하게 연결된다는 사실도 이해하게 되었다.

이번 프로젝트는 단순 시뮬레이터 구현을 넘어, 프로그램 실행의 구조적 원리를 이해하는 데 중요한 학습 경험이 되었으며, 이후 보다 복잡한 구조를 학습할 때 기준점이 되는 출발선이 될 것이라고 생각한다.

10. GitHub Repository

Full source code and test programs are available at:

https://github.com/carolyn0515/CPU